

**Programa de Ingeniería de Sistemas y Computación**

**Universidad del Quindío**

**Curso:** Programación Avanzada

**Guía:** 13

**Título:** IMPLEMENTACIÓN DE PRUEBAS UNITARIAS Y DE INTEGRACIÓN CON ENTORNOS SEGUROS

**Duración estimada:** 240 minutos

**Docentes:** Christian Andrés Candela

## OBJETIVO

Comprender qué son las pruebas unitarias y de integración, su importancia en el desarrollo de software y cómo implementarlas eficientemente en un ecosistema Spring Boot que cuente con Seguridad Transaccional (JWT) y Base de Datos Relacional (JPA/H2) bajo el esquema de Arquitectura Limpia.

## CONCEPTOS BÁSICOS

- **Pruebas Unitarias:** Verificación del comportamiento de unidades individuales de código (métodos o funciones) de manera aislada.
- **Pruebas de Integración:** Poner a trabajar el Sistema como un conjunto armónico y medir su red.
- **JUnit 5:** Framework predilecto de pruebas para la máquina virtual de Java.
- **Mockito:** Biblioteca especializada que nos permitirá simular ("Mapear/Clonar") a aquellos componentes con los que nuestra clase de prueba dependa, creando escenarios artificiales de éxito o catástrofe.
- **@WithMockUser** : Anotación clave de Spring Security para el entorno de testeo que permite simular credenciales válidas y circunvalar los JWT.

# CONTEXTUALIZACIÓN TEÓRICA

"Las pruebas no son solo para encontrar errores, son para prevenir que ocurran" - Martin Fowler

## Glosario Técnico

| Término         | Definición                                      | Ejemplo                      |
|-----------------|-------------------------------------------------|------------------------------|
| Prueba Unitaria | Verificación de unidades individuales de código | Probar un método de cálculo  |
| Mock            | Objeto simulado que reemplaza dependencias      | Simular una base de datos    |
| AAA Pattern     | Arrange-Act-Assert (Organizar-Actuar-Afirmar)   | Estructura básica de pruebas |
| Cobertura       | Porcentaje de código ejecutado por pruebas      | 80% de cobertura             |

## ¿Qué son las Pruebas Unitarias?

Las pruebas unitarias son pequeños tests que validan si una unidad específica de código funciona correctamente de manera aislada. Estas pruebas son esenciales porque:

- Detección temprana de errores:** Ayudan a identificar fallos en la lógica de negocio antes de que el código se integre con otros sistemas.
- Mejor mantenimiento del código:** Facilitan la refactorización sin riesgo de introducir errores.
- Mejoran la calidad del software:** Garantizan que cada componente funcione como se espera.

En el contexto de Spring Boot, las pruebas unitarias se enfocan en la validación de los métodos de los servicios, asegurando que las reglas de negocio se ejecuten correctamente empleando herramientas como JUnit y Mockito que anulan la necesidad real de una base de datos.

## Pruebas de Integración: El Puente entre Componentes

### Definición:

Las pruebas de integración validan la interacción correcta entre múltiples componentes o sistemas que funcionan conjuntamente. A diferencia de las unitarias, involucran:

- **Comunicación con bases de datos reales** (H2, MySQL)
- **Llamadas a servicios externos** o Filtros de Red (JWT, Cadenas de Seguridad).
- **Interacción entre capas** (Controlador → Servicio → Repositorio)

**Características clave:**

| Aspecto             | Descripción                                                                |
|---------------------|----------------------------------------------------------------------------|
| <b>Objetivo</b>     | Detectar fallos en interfaces e interacciones entre componentes integrados |
| <b>Velocidad</b>    | Moderada (segundos por prueba)                                             |
| <b>Alcance</b>      | Subconjunto del sistema (2+ componentes acoplados)                         |
| <b>Herramientas</b> | @SpringBootTest , @AutoConfigureMockMvc , bases de datos en memoria (H2)   |

## Tipos de Pruebas en Spring Boot

### 1. Pruebas de Capa Persistencia ( @DataJpaTest )

- Levanta **temporalmente** solo el contexto de Base de Datos para probar Repositorios y queries personalizadas. Ignora los Controladores.
- Ideal para H2 o testContainers.

### 2. Pruebas de Controladores ( @WebMvcTest )

- Levanta únicamente la capa Web (Controladores). Corta la comunicación a los servicios fingiéndolos a través de @MockitoBean .
- Útil para testear el cifrado o estatus de códigos REST (200 OK, 404 NOT FOUND).

### 3. Pruebas Holísticas/Integración ( @SpringBootTest )

- Arranca **TODA** la aplicación como si se pusiera en Producción.
- Altamente costosas para el procesador (y lentas), pero simulan la experiencia total.

## BUENAS PRÁCTICAS Y RECOMENDACIONES (PRINCIPIO F.I.R.S.T.)

- **Fast** (Rápidas): Las pruebas deben ejecutarse en milisegundos.
- **Isolated** (Aisladas): Cada prueba debe ser independiente y no depender del estado de otras pruebas.

- **Repeatable** (Repetibles): Producir idéntico resultado en cualquier entorno (Windows, Mac o Integración Continua).
- **Self-validating** (Auto-validables): Proporcionar inequívocamente mensajes como Aprobado/Fallido mediante `Asserts` sin mediación humana.
- **Timely** (Oportunas): Escribirse en paralelo con el código de producción.

## DESAFÍO: IMPLEMENTACIÓN SEGURA EN SUS CAPAS

En la Guía 12 cubrimos nuestra aplicación tras un muro Transaccional de Spring Security. Eso desencadenó que nuestros Servicios, Repositorios y Controladores ahora no operan si detectan amenazas. Implementaremos entonces técnicas vitales de validación basadas en Arquitectura Limpia para el módulo de Identidad ( `UsuarioEntity` ).

### 1 - Preparación Estructural

Al trabajar en Arquitectura Limpia, su árbol de pruebas ( `src/test/java` ) debe **clonar** las ramas o paquetes de la fuente de la aplicación ( `src/main/java` ).

1. En la carpeta `src/test/java` , dentro de su paquete maestro `co.edu.uniquindio.[proyecto]` , replique la estructura:
  - `infrastructure.persistence.jpa.repositories` -> Para nuestras pruebas de base de datos JPA (*Unitarias*).
  - `application.services.unit` -> Para pruebas con Mockito (*Unitarias*).
  - `infrastructure.rest.controllers.unit` -> Pruebas REST simuladas.
  - `infrastructure.rest.controllers.integration` -> Pruebas extremas de red real.
  - `infrastructure.config.setup.test` -> (Utilidades como fábricas de creación).
2. En la carpeta `src/test/resources` cree un archivo **`application.properties`** . Todo lo que emane acá sobreescribe al proyecto de Producción solo durante las Pruebas:

```
# H2 en memoria destruible por cada test para garantizar aislamiento (aislamiento total)
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
spring.datasource.driverClassName=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
```

```
# Seguridad HMAC
jwt.secret=MiClaveSecretaSuperSeguraParaJWT2026EnProgramacionAvanzadaUniquindio
jwt.expiry=5
```

## 2 - Fábrica de Datos Utilitaria

Dado que en las Pruebas a cada instante el Sistema levantará de cero y quemará entidades en base de datos para ensayar, construiremos fábricas estáticas veloces para no llenar de entidades *basura* nuestras clases lógicas.

1. En la paquetería de Utilidades `setup.test` cree `UsuarioTestDataLoader` :

```

import co.edu.uniquindio.[proyecto].infrastructure.persistence.jpa.entity.UsuarioEntity
import co.edu.uniquindio.[proyecto].infrastructure.persistence.jpa.entity.RolSeguridadE
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.UUID;

public class UsuarioTestDataLoader {

    public static final PasswordEncoder encoder = PasswordEncoderFactories.createDelega

    public static UsuarioEntity usuarioAdminBase() {
        UsuarioEntity admin = new UsuarioEntity();
        admin.setId(UUID.randomUUID().toString());
        admin.setEmail("admin@test.com");
        admin.setPassword(encoder.encode("adminpass"));
        admin.setRol(RolSeguridadEnum.ADMIN);
        return admin;
    }

    public static UsuarioEntity usuarioUserBase() {
        UsuarioEntity user = new UsuarioEntity();
        user.setId(UUID.randomUUID().toString());
        user.setEmail("user@test.com");
        user.setPassword(encoder.encode("userpass"));
        user.setRol(RolSeguridadEnum.USER);
        return user;
    }
}

```

### 3 - PRUEBAS LIMITADAS AL REPOSITORIO JPA ( @DataJpaTest )

Evaluaremos puramente la trinchera SQL. Esta prueba cortará Servicios, Controladores y Seguridad. Simplemente revisará que el Repositorio de base de datos grabe y haga Match con nuestras queries.

1. En el paquete de pruebas `infrastructure.persistence.jpa.repositories` cree `UsuarioJpaRepositoryTest` :

```

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest;
import org.springframework.boot.jpa.test.autoconfigure.TestEntityManager;
import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest // Magia para configurar H2 volátil de inmediato
class UsuarioJpaRepositoryTest {

    @Autowired
    private TestEntityManager entityManager; // Componente nativo de JPA Testing

    @Autowired
    private UsuarioJpaRepository usuarioRepository;

    private UsuarioEntity testAdmin;

    @BeforeEach
    void setUp() {
        // [ARRANGE] Limpiar e inyectar fresco
        usuarioRepository.deleteAll();
        testAdmin = UsuarioTestDataLoader.usuarioAdminBase();
        entityManager.persistAndFlush(testAdmin); // Grabamos en H2 explícitamente para
    }

    @Test
    void testBusquedaPorCorreoExitosa() {
        // [ACT]
        var optUser = usuarioRepository.findByEmail("admin@test.com");

        // [ASSERT] Valida si nuestra firma en el Repositorio de Spring Data hace Magia
        assertTrue(optUser.isPresent());
        assertEquals(RolSeguridadEnum.ADMIN, optUser.get().getRol());
    }

    @Test
    void testBusquedaPorCorreoInexistente() {
        var optUser = usuarioRepository.findByEmail("fantasma@test.com");
        assertTrue(optUser.isEmpty());
    }
}

```

## 4 - PRUEBAS DE SERVICIO PURO UNITARIO (Mockito)

Acá cambia el panorama. El servicio contiene lógica de negocio ( `SecurityServiceImpl` , `SolicitudServiceImpl` ), pero **NO** queremos consumir tiempo escribiendo de verdad en Bases de datos para probar transacciones lógicas. Para eso usamos clones de sombra ( `Mocks` ).

1. En `application.services.unit` evalúe sus lógicas usando Extensión Mockito (Evadimos Inicializar todo Spring aquí).



```

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import static org.mockito.Mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;

@ExtendWith(MockitoExtension.class) // Inicializador de entorno ligero, sin base de dat
class SecurityServiceImplTest {

    @Mock
    private UsuarioJpaDataRepository repositoryMock; // Clonamos la Base de datos

    @Mock
    private JwtTokenProvider jwtProviderMock; // Clonamos generadores costosos

    @Mock
    private AuthenticationManager authManagerMock; // Anulamos los engranajes densos de

    @InjectMocks
    private SecurityServiceImpl securityService; // La victima que usará a los de arrib

    private UsuarioEntity baseUsuarioEntity;

    @BeforeEach
    void setUp() {
        // Obtenemos los Mocks puramente lógicos creados previamente
        baseUsuarioEntity = UsuarioTestDataLoader.usuarioUserBase();
    }

    @Test
    void testLoginRetornaJWTResponseYNoCaiga() {
        // [ARRANGE – Mentiras Verdaderas]
        // Obligamos a la simulación del Gestor a que siempre diga Que Sí funciona.

        Authentication dummyAuth = new UsernamePasswordAuthenticationToken(
            baseUsuarioEntity.getEmail(),
            "userpass",
            List.of(new SimpleGrantedAuthority(baseUsuarioEntity.getRol().name()))
        );
    }

```

```

when(authManagerMock.authenticate(any(UsernamePasswordAuthenticationToken.class)
when(jwtProviderMock.generateTokenAsString(anyString(), anyList(), any(), any())

// [ACT]
var response = securityService.login(new LoginRequest(baseUsuarioEntity.getEmail()

// [ASSERT]
assertNotNull(response);
assertEquals("Bearer", response.type());
assertEquals("eyJzbn21lVG9rZW4i...", response.token());

// Verificamos matemáticamente que el Servicio en efecto llamó a los repositori
verify(authManagerMock, times(1)).authenticate(any());
verify(jwtProviderMock, times(1)).generateTokenAsString(anyString(), anyList(),
}
}

```

## 5 - CONTROLADORES REST UNITARIOS (Traspassando La Seguridad)

Este tipo de prueba ( @WebMvcTest ) solo convoca su capa REST Controller (Endpoint) para medir JSONs y Status de respuesta (201, 200).

**EL GRAN PROBLEMA** radica en que Spring Security se inyectará activamente y devolverá Error 401 Unauthorized .

Para evitarlo recurrimos a @WithMockUser , inyectando una autorización temporal de mentiras a la red.

1. En el paquete de pruebas infrastructure.rest.controllers.unit cree la prueba:

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequest

// Configurador. Corta Servicios Reales y Base de Datos Reales.
@WebMvcTest(controllers = PerfilController.class) // Endpoint ficticio de ejemplo
@Import({SecurityConfig.class, JwtConfig.class}) // IMPORTANTE: Cargar la configuración
class PerfilControllerUnitTest {

    @Autowired
    private MockMvc mockMvc; // Emulador de navegadores Postman a altísima velocidad.

    @MockitoBean
    private PerfilApplicationService perfilServiceMock; // Simulamos su capa de lógica

    @Test
    void testAccederAUnEndpointProtegidoYRetornar200() throws Exception {

        // [ARRANGE]
        when(perfilServiceMock.obtenerMiPerfil()).thenReturn(new MiPerfilRecord("Soy Fe

        // [ACT & ASSERT]
        // Usamos el inyector post-processor jwt() para fingir un token válido en la ca
        mockMvc.perform(get("/api/perfiles/me")
            .with(jwt().jwt(builder -> builder.subject("ficticio@user.com"))
                .authorities(new org.springframework.security.core.authority
            ))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.bio").value("Soy Feliz")); // Buscamos en el JSO
    }

    @Test
    void testUsuarioSinSelloAnotaHTTP401() throws Exception {
        // [NO WITH MOCK USER AQUI]
        mockMvc.perform(get("/api/perfiles/me"))
            .andExpect(status().isUnauthorized()); // El escudo de JWT que hicimos
    }
}

```

## 6 - PRUEBAS DE END-TO-END (INTEGRACIÓN MASIVA)

Para las clases anotadas con `@SpringBootTest` la aplicación se encenderá completa, base de datos completa. No simularemos nada, es nuestra última barrera de QA.

Puesto que todo opera como Producción, `@WithMockUser` se queda pequeño en algunas interacciones densas. Aquí es recomendable usar un utilitario real que dispare un Login contra nuestra BD H2 de prueba en tiempo de ejecución, extraiga la llave JWT, y se la pase por medio del String Header de Autorización.

1. Ubique una clase utilitaria temporal en `infrastructure.config.setup.test` de validación global HTTP:

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.jayway.jsonpath.JsonPath;
import org.springframework.test.web.servlet.MockMvc;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

public class LoginIntegrationTestUtil {
    public static String obtenerTokenIntegro(String correo, String contrasenaPlana, Moc
        var req = new LoginRequest(correo, contrasenaPlana);
        var result = mvc.perform(post("/api/auth/login")
                                .contentType("application/json")
                                .content(objMapper.writeValueAsString(req)))
                        .andExpect(status().isOk())
                        .andReturn();

        var bodyResponseJson = result.getResponse().getContentAsString();
        return JsonPath.parse(bodyResponseJson).read("$.token").toString();
    }
}
```

2. Implementación de una Prueba E2E en `infrastructure.rest.controllers.integration` :

```

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import com.fasterxml.jackson.databind.ObjectMapper;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
public class IntegracionEndToEndTest {

    @Autowired
    private MockMvc mockMvc;

    // Instanciamos el mapper manualmente para evitar errores de contexto parcial en Sp
    private ObjectMapper objectMapper = new ObjectMapper();

    @Autowired
    private UsuarioJpaDataRepository repository;

    private UsuarioEntity semillaGuardada;

    @BeforeEach
    void setupRealDatabase() {
        repository.deleteAll();
        semillaGuardada = repository.save(UsuarioTestDataLoader.usuarioAdminBase())
    }

    @Test
    void testTravesiaCompletaDePeticionAEndpointSeguro() throws Exception {

        // 1. Efectuamos el puente real del log-in
        String tokenVivo = LoginIntegrationTestUtil.obtenerTokenIntegro(
            semillaGuardada.getEmail(),
            "adminpass", // Contraseña nativa pre-encryptada
            mockMvc, objectMapper
        );

        // 2. Comisionamos Postman simulado atacando endpoints duros
        mockMvc.perform(get("/api/usuarios/"+semillaGuardada.getId())
            .header("Authorization", "Bearer " + tokenVivo)
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.email").value("admin@test.com")));
    }
}

```

```
}  
}
```

## EVALUACIÓN Y CONOCIMIENTO

Se espera que el estudiante al terminar la guía alcance las habilidades técnicas para:

- Crear escenarios en capas independientes resguardando tiempos de máquina CPU.
- Implementar Mockito contra Capas Persistentes.
- Superar barreras de Security y JWT empleando utilitarios propios en pruebas de Integración pura ( `LoginIntegrationTestUtil` ).
- Entender el concepto de `Arrange-Act-Assert` ante pruebas de status code reales HTTP.

## REFERENCIAS BIBLIOGRÁFICAS

- Spring Boot Data JPA Testing: <https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing>
- [JUnit 5 - Mockito Baeldung Guideline](#)
- [Spring WebMvcTest and Security contexts](#)
- [Testing Best Practices - Martin Fowler](#)